ARIAN

Azerbaijan Wildfire & Climate Forecast

Project Blueprint v2 · Process-aligned team design · April 2026

What changed in v2

This is a rewrite of the original blueprint after auditing the actual data you have. Key differences: **(1)** team roles are now split by **process step** (fetch → clean → feature → climate → forecast → risk → map) rather than by abstract layer, so handoffs are concrete and testable. **(2)** the training window has been corrected — your Open-Meteo bundle only spans 2024-03 to 2026-04, which is why the old "train 2020-23" plan was infeasible; we extend it via free ERA5 reanalysis. **(3)** the deliverables are tied to a specific **notebook** (**ARIAN_wildfire_pipeline.ipynb**) that actually runs end-to-end on the provided files, and a **fetch script** (**fetch_era5_history.py**) that one team member runs once to extend the dataset.

§1 Data audit — what we actually have

Before designing roles we measured every dataset you provided. Critical finding: **weather data covers only April 2024 to April 2026** in the Open-Meteo bundle. Fires go 2020-2024. That's ~9 months of overlap, too narrow to train and validate a fire classifier the way the original blueprint described. **We fix this by extending weather back to 2015 via ERA5 reanalysis** (free, no API key, Open-Meteo Archive API). This is a one-time download handled by one script.

Dataset	Date range	Granularity	Status in v2
Open-Meteo daily + hourly (bundle)	2024-03-31 → 2026-04-22	Daily + hourly, 16 cities	Primary source, 2024→ present
ERA5 reanalysis (Open-Meteo Archive)	2015-01-01 → yesterday	Daily, 16 cities + 8 forest pts	Added by fetch_era5_history.py
NASA FIRMS VIIRS-SNPP	2020-01-01 → 2024-12-31	Per-detection, 21 k rows	Fire labels (type=0 only)
MODIS NDVI 16-day	2020-01-01 → 2026-03-22	16-day, 16 cities	Vegetation stress feature
WorldPop GHS-POP 100 m	2020 → 2026	Annual TIFs	Human ignition proxy
ESA WorldCover	static (1 TIF)	10 m raster	Landcover class per node
OSM road network	latest snapshot	PBF	Road-distance feature (future)

§2 The 7 process steps → 5 team roles

The pipeline has seven process steps. We map them to five roles so every step has a clear owner, and every pair of adjacent steps has a single, well-defined artifact passing between them.

#	Process step	Owner role	Hand-off artifact
1	Fetch & version raw data	DE — Data Engineer	ARIAN/ folder + fetch_era5_history.py output
2	Clean, validate, join	DC — Data Engineer (same role)	master daily (node × date) frame
3	Feature engineering (lags, FWI, NDVI, landcover)	FE — Feature Engineer	feature_matrix.parquet + feature_schema.json

#	Process step	Owner role	Hand-off artifact
4	Climate anomaly analysis (is 2025/26 hotter?)	CA — Climate Analyst	arian_climate_anomaly_*.csv + figures
5	30-day weather forecasting	CA — Climate Analyst (same role)	arian_weather_forecast_30day.csv
6	Wildfire risk model + 30-day risk forecast	ML — ML Engineer	calibrated_model.pkl, arian_30day_risk_*.csv
7	Interactive map + storytelling	MF — Map / Frontend Engineer	arian_map.html

Note: steps 1–2 are one role (Data Engineer) because validation is inseparable from loading — you need to see the data to know what's broken. Steps 4–5 are one role (Climate Analyst) because they share the same SARIMAX / bootstrap machinery and both consume the same feature matrix without needing the fire labels. Steps 6 and 7 are kept separate because the ML work is numerical and the map work is design/UX; conflating them typically produces pretty maps of bad numbers or correct numbers shown badly.

§3 Role cards

DE Data Engineer

Branch: `data-ingestion` · **Owns steps:** 1, 2

Goal

Get every data source into one clean, validated, versioned folder so no one else ever has to worry about file formats, bad dates, or wrong coordinates again. This role is the foundation — if it's wrong, everything downstream is wrong.

Deliverables

- Complete ARIAN/ data folder exactly as documented (see §4).
- fetch_era5_history.py script run once per month to keep the ERA5 baseline fresh.
- data_validator.py that fails loudly if a source has coordinate drift, column rename, >5 % missing, or >3 consecutive NaN days.
- A 1-page README inside ARIAN/ documenting every file, its source URL, its license, and how to re-fetch it.
- nasa_firms_refresh.py script that downloads any new VIIRS-SNPP data for Azerbaijan since the last run.

Key files they own

- `ARIAN/OpenMeteo/aze_regional_daily_weather_extended.parquet`
- `ARIAN/OpenMeteo/aze_regional_daily_weather.csv`
- `ARIAN/NASA_FIRMS/viirs-snpp_YYYY_Azerbaijan.csv (one per year)`
- `ARIAN/EarthEngine/AZE_City_NDVI_2020-2026.csv`
- `scripts/fetch_era5_history.py`
- `scripts/data_validator.py`

Main tools

requests / httpx · pandas · pyarrow · rasterio · cdsapi (if ERA5 via CDS) · Open-Meteo Archive API (preferred, no key)

Weekly check-in questions

- Does data_validator.py exit 0 on every source?
- Did we add any new grid point this week, and if so is it in every downstream file?
- When was FIRMS last refreshed? (Should be ≤ 7 days old.)

FE Feature Engineer

Branch: `feature-engineering` · **Owns steps:** 3

Goal

Turn the raw per-(node, date) frame into a model-ready feature matrix. Every column must have a clear, one-sentence physical interpretation — no “magic” features. Think of this role as a physicist building a fire-behaviour model by hand.

Deliverables

- ~45-column feature matrix per (node, date), covering: weather (raw + 1/3/7-day lags + 7-day rolling mean & std), fuel-moisture (Canadian FWI family: FFMC, DMC, DC, ISI, BUI, FWI), vegetation (NDVI daily-

resampled), terrain/static (landcover one-hot, pop density), calendar (day-of-year sin/cos, month), and weighted dry-streak (exponential decay half-life = 5 days).
- feature_schema.json listing name, dtype, units, physical meaning, and source column for every feature.
- A unit-test suite (tests/test_features.py) that asserts: no NaN in >95 % of rows after the first 7 days per node, FWI values in 0–100 range, dry_streak_decay in [0, 1].
- A one-page memo explaining why each FWI sub-code is computed day-by-day per node and not vectorised — the iterative formulation is the point.

Key files they own

- `notebooks/ARIAN_wildfire_pipeline.ipynb (cells in §6 “Feature engineering”)`
- `src/arian/features.py (add_features + compute_fwi refactored into importable module)`
- `outputs/feature_matrix.parquet`
- `outputs/feature_schema.json`

Main tools

pandas · numpy · scipy · rasterio (for NDVI/landcover/pop sampling) · pytest

Weekly check-in questions

- Are any features strongly correlated (|r|>0.95) that we could drop?
- What's the SHAP ranking of the newest feature we added? If it's bottom-quartile, remove it.
- Does the feature matrix regenerate byte-identically from scratch?

CA Climate Analyst

Branch: `climate-and-forecasting` · **Owns steps:** 4, 5

Goal

Answer the two “big picture” questions: (a) is this year hotter than the last decade, with what statistical confidence? (b) what will the weather look like for the next 30 days? Own both the historical climatology and the short-range forecast — they share the same data and the same math.

Deliverables

- Per-(city, calendar-month) anomaly analysis: z-score vs 2015-2024 decade baseline, 95 % bootstrap CI of the baseline mean, flag for “above upper CI”.
- Country-wide yearly summary (mean anomaly °C, % of city-months above upper CI) with a one-sentence headline.
- Per-city Mann-Kendall trend test on annual T_max with Sen's slope estimator. Report how many cities reach p<0.05.
- 30-day forecast per city for T_max, T_min, precipitation using SARIMAX(1,1,1)(0,0,0,7) + K=3 Fourier annual terms. 90 % prediction intervals.
- Walk-forward validation (last 60 days hold-out, 30-day horizon) — report per-variable MAE and any city where MAE > 3 °C.
- Fallback model (harmonic regression + AR(7)) when statsmodels is unavailable.

Key files they own

- `outputs/arian_climate_anomaly_country.csv`
- `outputs/arian_climate_anomaly_detail.csv`
- `outputs/arian_weather_forecast_30day.csv`
- `figures/anomaly_heatmap_YYYY.png`

- `figures/weather_forecast_examples.png`

Main tools

statsmodels (SARIMAX) · scipy.stats · sklearn.linear_model.Ridge (fallback) · numpy bootstrap

Weekly check-in questions

- What's our T_max 30-day MAE this week? (Target: < 2.5 °C country-mean.)
- How many cities now have a significant warming trend (Mann-Kendall p<0.05)?
- Did the 2026 anomaly direction flip? Explain what changed.

ML ML Engineer (Wildfire Risk)

Branch: `ml-pipeline` · **Owns steps:** 6

Goal

Produce a calibrated probability of at least one fire within ~25 km of every (node, day) in the next 30 days, and explain which features drove each prediction. Obsess over calibration, not just accuracy — a 75 % risk must mean “75 out of 100 similar days had a fire.”

Deliverables

- TimeSeriesSplit(5) training on the full labelled window (2020-2024 once ERA5 is fetched, else Apr–Dec 2024).
- SMOTE + Tomek Links for class imbalance (positives are 3–5 % of rows).
- Optuna hyperparameter search (~30 trials, optimise PR-AUC on the inner validation fold).
- CalibratedClassifierCV(method='isotonic', cv=3) wrapper so probabilities are honest.
- Evaluation on the outer 15 % chronological hold-out: ROC-AUC, PR-AUC, Brier score, F1, reliability diagram, confusion matrix.
- SHAP TreeExplainer — beeswarm plot, per-prediction top-3 drivers for the map popups.
- 30-day forward risk: feed the Climate Analyst's forecast into the calibrated model, day-by-day, recomputing lag / rolling / FWI features as we walk forward. This is the only correct way.
- arian_eval_report.json with every metric versioned by git commit SHA.

Key files they own

- `outputs/models/calibrated_model.pkl`
- `outputs/arian_eval_report.json`
- `outputs/arian_30day_risk_daily.csv`
- `outputs/arian_30day_risk_summary.csv`
- `outputs/shap_global_ranking.csv`
- `figures/model_diagnostics.png`
- `figures/shap_beeswarm.png`

Main tools

xgboost · scikit-learn (CalibratedClassifierCV, TimeSeriesSplit) · imbalanced-learn (SMOTE+Tomek) · optuna · shap

Weekly check-in questions

- What are this week's PR-AUC and Brier score on the hold-out? (Reject PRs that make either worse.)
- Has the top SHAP driver changed? If so, does it still make physical sense?
- Are we over-predicting in any specific city? (Look at per-city recall vs precision.)

MF Map / Frontend Engineer

Branch: `frontend-map` · **Owns steps:** 7

Goal

Turn the numbers into something a 60-second glance can interpret. Dark-themed, information-dense, but not noisy. The map is the only deliverable most stakeholders will ever look at.

Deliverables

- Single-file outputs/arian_map.html — no build step, no CDN dependencies beyond Leaflet.
- Dark CartoDB tile layer + heatmap overlay colored by peak 30-day risk per node.
- Rich per-node popup: risk gauge (SVG bar), peak date, top driver, 30-day sparkline, climate-anomaly badge (Δ vs decade mean in °C).
- Fixed-position top banner: “N nodes ≥ HIGH · N nodes ≥ EXTREME” with forecast window.
- Layer toggle, fullscreen, measure tool, consistent color legend (LOW/MOD/HIGH/EXTREME).
- (Optional) a Cloudflare Pages or Render deploy URL so the map is reachable from any phone.
- (Stretch) Service Worker + manifest.json so the map is installable as a PWA and works offline.

Key files they own

- `outputs/arian_map.html`
- `frontend/templates/popup_template.svg (for design iteration)`
- `frontend/deploy/netlify.toml or render.yaml`

Main tools

folium · leaflet · SVG · (stretch) PWA Service Worker

Weekly check-in questions

- Does the map load in <3 s on a mid-tier phone?
- Is the color legend consistent with the thresholds ML Engineer reports?
- Did we update the banner thresholds this week, and does the legend still match?

§4 Folder layout & artefact contract

Every role writes to its own subfolder, and a downstream role never reads a different role's working files — only the promised artefacts. This makes parallel work possible.

Path	Purpose
ARIAN/	Raw data (read-only after DE owns it)
ARIAN/NASA_FIRMS/	VIIRS-SNPP yearly CSVs
ARIAN/OpenMeteo/	Hi-res bundle + ERA5 extended parquet
ARIAN/EarthEngine/	NDVI CSV + landcover TIF
ARIAN/Population/	WorldPop yearly TIFs
scripts/	fetch_era5_history.py, data_validator.py
notebooks/ARIAN_wildfire_pipeline.ipynb	The one canonical end-to-end pipeline
src/arian/features.py	FE role — importable feature fns
src/arian/models.py	ML role — model + training fns
src/arian/forecast.py	CA role — SARIMAX + bootstrap fns
outputs/	All final CSVs, figures, pickles, map
outputs/figures/	Every plot the notebook produces
outputs/models/calibrated_model.pkl	The model MF embeds into the map
outputs/arian_map.html	The final deliverable stakeholders open

§5 Data sources — everything is free

The project uses only freely-licensed data. The fetch_era5_history.py script needs no registration; everything else is a manual download documented below.

Dataset	URL / how to get	License	Owner role
ERA5 reanalysis (daily, 1940→)	fetch_era5_history.py → archive-api.open-meteo.com/v1/archive	CC-BY 4.0	DE
Open-Meteo Forecast (hi-res 2024→)	api.open-meteo.com — no key	CC-BY 4.0	DE

Dataset	URL / how to get	License	Owner role
NASA FIRMS VIIRS-SNPP	firms.modaps.eosdis.nasa.gov (free reg'n)	Public domain	DE
MODIS NDVI (16-day)	Earth Engine export (already bundled)	NASA open	DE
WorldPop GHS-POP	worldpop.org (free download)	CC-BY 4.0	DE
ESA WorldCover 2021	esa-worldcover.org (free download)	CC-BY 4.0	DE
OSM road network (Azerbaijan)	download.geofabrik.de/asia/azerbaijan.html	ODbL	DE
SRTM DEM 30 m (optional)	earthexplorer.usgs.gov (free reg'n)	Public domain	DE
EFFIS historical fires (optional)	effis.jrc.ec.europa.eu	EU open data	DE

§6 Milestones

Five-week plan. Every role has a deliverable every week. No role waits on another for more than one working day.

Week	DE	FE	CA	ML	MF
W1	fetch_era5 runs clean. data_validator ok.	Feature schema v1 drafted.	Bootstrap CI code reviewed.	Baseline XGB on the Apr–Dec 2024 overlap.	Empty Folium map with 24 dots.
W2	All sources documented in ARIAN/README.md.	FWI codes integrated + unit-tested.	Climatology heatmap first version.	SMOTE+Tomek + TimeSeriesSplit working.	Dark tiles, layers, banner design.
W3	Weekly FIRMS refresh automated.	feature_matrix.parquet v1 frozen.	SARIMAX 30-day forecasts per city.	CalibratedClassifierCV + reliability diagram.	Popup with gauge + sparkline.
W4	Monthly ERA5 top-up script.	feature_schema.json published.	Mann-Kendall trend +	SHAP beeswarm + top driver export.	Popup with top driver + anomaly badge.

Week	DE	FE	CA	ML	MF
			headline text.		
W5	Handover doc + scripts tagged v1.0.	Feature docs for v1.0.	CA headlines approved for pitch.	calibrated_model.pkl v1.0.	arian_map.html deployed + shared link.

§7 How to run everything

Once — extend the weather record

```
pip install pandas pyarrow requests tqdm
```

```
python scripts/fetch_era5_history.py
```

This writes ARIAN/OpenMeteo/aze_regional_daily_weather_extended.parquet (~10 MB). Takes 3–5 min.

Every time — run the pipeline

```
pip install pandas pyarrow matplotlib seaborn scipy scikit-learn statsmodels
xgboost imbalanced-learn shap folium rasterio optuna tqdm
```

```
jupyter lab notebooks/ARIAN_wildfire_pipeline.ipynb  →  Cell → Run All
```

Total runtime ~ 10–15 min on a laptop. Outputs land in outputs/. Open outputs/arian_map.html in any browser.

If anything fails

- ModuleNotFoundError → pip install the missing one (listed above).
- “Extended ERA5 NOT found” warning → you skipped step 1; the notebook still runs but the climate comparison is a no-op.
- Memory error in FWI cell → restart kernel, close other notebooks. FWI needs one dataframe-scan per node, so ~18 k rows fit in ~200 MB.
- Map looks empty → open the browser console; folium scripts sometimes need allow-http on file://.